

Mean Reciprocal Rank (MRR), Worked Out by Hand

The metric for “how high was the *first* right answer?” — it takes the reciprocal of the rank of the first relevant result and averages over queries. Computed across a three-query evaluation set.

What this document does. It defines reciprocal rank, computes it for three queries whose first relevant result lands at different positions, averages them into MRR, and explains exactly what this metric rewards (and ignores). Numbers from Appendix A.

When to use it: MRR is the cheap, binary-label cousin of nDCG (#31) — ideal when each query has essentially one right answer and you only care how fast you surface it.

1 What it measures

For tasks where each query has one (or one principal) correct answer — fact lookup, “go to” navigation, known-item search — the only thing that matters is *how far down the list the user had to look to find it*. MRR captures exactly that and nothing else.

Intuition

Reciprocal rank is the “how many results did I have to scan?” metric, inverted so bigger is better. First result correct? Score $1/1 = 1$. Had to scroll to the third? $1/3$. The reciprocal makes the top positions matter enormously and the deep ones barely at all — the difference between rank 1 and 2 (1.0 vs 0.5) dwarfs the difference between rank 9 and 10 (0.111 vs 0.100). That matches how users actually behave: almost nobody looks past the first few results.

2 The formula

For each query i , let rank_i be the position of the *first* relevant result. The reciprocal rank is $1/\text{rank}_i$, and MRR averages it over Q queries:

$$\text{MRR} = \frac{1}{Q} \sum_{i=1}^Q \frac{1}{\text{rank}_i} \in (0, 1].$$

If no relevant result appears (within the cutoff), that query contributes 0. Only the *first* hit counts — anything after it is ignored entirely.

3 Worked by hand

Three queries; we record where the first relevant result appeared in each ranked list:

query	rank of first relevant	reciprocal rank
q_1	2	$1/2 = 0.5000$
q_2	1	$1/1 = 1.0000$
q_3	4	$1/4 = 0.2500$

$$\text{MRR} = \frac{0.5000 + 1.0000 + 0.2500}{3} = \frac{1.7500}{3} = \boxed{0.5833}.$$

q_2 nailed it at rank 1; q_1 was one slot off; q_3 buried the answer at rank 4, contributing only 0.25. The mean, 0.58, summarises “on average the first right answer sits around rank 1.7.”

Mini-example · why rank 4 is punished so hard

q_3 ’s answer at rank 4 scores 0.25 — but if it had been at rank 2 it would score 0.50, *doubling* its contribution. Moving from rank 4 to rank 3 only buys 0.083 ($0.25 \rightarrow 0.333$); moving from rank 2 to rank 1 buys a full 0.50. The reciprocal front-loads all the reward into the very top positions, which is the whole point: for known-item search, being second is much worse than being first, and being tenth is barely better than being absent.

4 What MRR ignores — and when that is wrong

- **It only sees the first hit.** If a query has several relevant documents, MRR ignores all but the earliest — it cannot reward a system for finding *more* of them. When recall of multiple relevants matters, use MAP (#30).
- **Relevance is binary.** A “perfect” and a “barely relevant” first hit score identically. When results have grades, use nDCG (#31).
- **It is blind below the first hit.** Two systems with the first relevant at rank 1 tie at 1.0 even if one then lists garbage and the other lists more gold.

Within its niche — one right answer per query, binary labels, only the top spot matters — MRR is the simplest and most apt metric. Outside it, reach for MAP or nDCG.

Equation cheat-sheet

$$\text{MRR} = \frac{1}{Q} \sum_{i=1}^Q \frac{1}{\text{rank}_i}, \quad \text{rank}_i = \text{position of first relevant for query } i$$

only the first hit counts; binary relevance; multiple relevants $\Rightarrow$ MAP; grades $\Rightarrow$ nDCG

Appendix A · Reference implementation

```
def mrr(first_relevant_ranks):
    return sum(1 / r for r in first_relevant_ranks) / len(first_relevant_ranks)

mrr([2, 1, 4])      # (0.5 + 1.0 + 0.25) / 3 = 0.5833

# from full ranked lists (1 = relevant):
def reciprocal_rank(rels):
    for k, r in enumerate(rels, 1):
        if r: return 1 / k
    return 0.0
```

Internal learning material. Numbers reproducible from the appendix code. v1.0